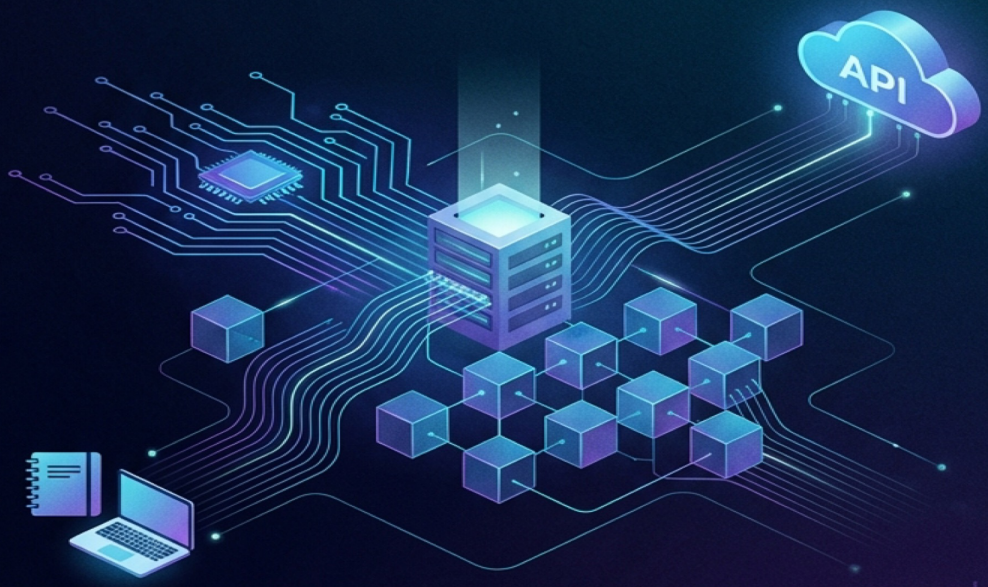
Blink Arquitetura RAG



Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Blink Arquitetura RAG

Alberto Souza

2026-02-23

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Contents

Introdução: De notebooks locais a APIs de IA em produção i

FastAPI e o padrão para APIs de IA: de notebook local a endpoint HTTP 1

Pydantic como guardião dos dados: validação automática na entrada e na saída 2

Arquitetura em camadas: config, models e services como pilares do projeto 4

Embeddings como serviço isolado: encapsulando a geração de vetores 5

Busca híbrida em camadas: dense, sparse e ColBERT trabalhando juntos 6

O endpoint de search: transformando busca vetorial em API acessível 8

Do search ao RAG: adicionando geração de respostas com LLM 9

Injeção de dependência entre services: composição em vez de acoplamento 11

Routers como módulos plugáveis: refatorando para escalar 12

Prompt engineering desacoplado: separando instruções da lógica de negócio 14

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Resumo Final 16
Sobre o Curso 18

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Introdução: De notebooks locais a APIs de IA em produção

Imagine que você construiu um pipeline de inteligência artificial incrível no seu notebook local. Ele recupera dados, gera embeddings, faz busca semântica e até produz respostas elaboradas com um LLM. Mas como transformar isso em algo que outras pessoas possam usar? Como sair do protótipo para uma aplicação real, acessível por qualquer sistema via HTTP?

O curso "Arquitetura RAG" de Daniel Romero responde exatamente a essa pergunta. Partindo de um sistema de ingestão de dados financeiros que já funciona com chunking semântico e busca híbrida no Qdrant, o curso guia você na construção de uma API completa usando FastAPI – desde a estrutura modular do projeto até a integração com um LLM para gerar respostas contextualizadas no padrão RAG (Retrieval-Augmented Generation).

Nos próximos minutos, você vai aprender:

- Como estruturar uma aplicação FastAPI de forma modular e profissional, separando responsabilidades em camadas
- Como usar Pydantic para validação automática de dados de entrada e saída
- Como encapsular a lógica de embeddings em um serviço isolado e reutilizável
- Como implementar busca híbrida no Qdrant combinando dense, sparse e ColBERT embeddings
- Como construir um endpoint RAG que usa chunks recuperados como contexto para um LLM
- Como refatorar rotas usando routers do FastAPI para manter o código limpo

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

- Como desacoplar prompts da lógica de negócio para facilitar a iteração de prompt engineering
- Como pensar na arquitetura como uma plataforma extensível, onde cada funcionalidade vira um novo endpoint

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

FastAPI e o padrão para APIs de IA: de notebook local a endpoint HTTP

No universo Python e da engenharia de IA, o FastAPI se consolidou como o padrão para construir APIs. A proposta é simples e poderosa: transformar aquele código Python que roda em um notebook local em algo que outras pessoas possam acessar através de requisições HTTP. Você tem um endpoint que recebe dados, processa com a sua lógica de IA e retorna uma resposta estruturada.

O diferencial do FastAPI está na combinação de simplicidade e poder. O framework é minimalista – o arquivo principal da aplicação pode ter poucas linhas. Ele roda sobre o Uvicorn, um servidor ASGI (Asynchronous Server Gateway Interface), que é o padrão de interface entre servidores web e aplicações Python assíncronas. Além disso, o FastAPI gera documentação interativa automaticamente via Swagger UI, acessível no endpoint `/docs`, onde você pode testar seus endpoints diretamente no navegador.

> *"A ideia aqui e bem simples, que e transformar aquele codigo Python, ou aquele codigo que esta em um notebook local, em algo que outras pessoas possam acessar atraves de requisicoes HTTP."*

Ao iniciar um projeto com FastAPI, resista à tentação de colocar tudo em um único arquivo. Desde o início, divida seu código em arquivos separados – main, router, endpoint, schemas. Essa organização modular, comum em projetos reais, facilita a manutenção e a extensão da aplicação conforme novos endpoints são adicionados.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Pydantic como guardião dos dados: validação automática na entrada e na saída

Uma API robusta precisa garantir que os dados que entram e saem estejam no formato correto. No ecossistema FastAPI, quem cuida disso é o Pydantic. Ao criar classes que herdam de `BaseModel`, você define esquemas de dados com tipos específicos para cada campo. Se uma requisição chegar com dados que não batem com o formato esperado, o Pydantic valida automaticamente e rejeita, retornando uma mensagem de erro detalhada com status 422.

O poder dessa abordagem fica claro quando você percebe que não precisa escrever nenhum código de validação manual. Quando você declara um parâmetro de função com o tipo de um modelo Pydantic, o FastAPI automaticamente extrai o JSON do corpo da requisição HTTP, valida os campos, converte os valores para os tipos adequados e passa um objeto já validado e pronto para uso. Na saída, o `response_model` do decorator garante que a resposta também segue o formato esperado, gerando documentação OpenAPI automaticamente e serializando o objeto Python para JSON.

> *"O Pydantic e uma biblioteca de validacao de dados que permite definir esquemas usando classes Python. Quando voce cria uma classe que herda de BaseModel, voce esta dizendo exatamente qual formato de dados voce espera receber, com os tipos especificos para cada campo."*

Use Pydantic não apenas para validar a entrada, mas também para definir o formato da saída com `response_model`. Isso cria um contrato claro entre a API e seus consumidores: quem chama a API sabe exatamente o que vai receber, e o FastAPI garante que

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

a resposta nunca fuja do formato definido.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Arquitetura em camadas: config, models e services como pilares do projeto

Antes de escrever a primeira linha de código da API, é fundamental pensar na estrutura do projeto. A abordagem adotada no curso organiza tudo em três diretórios principais: `config` para centralizar configurações, `models` para os modelos Pydantic que definem a estrutura dos dados e `services` para encapsular a lógica de busca e embeddings. Cada diretório vira um pacote Python com seu `__init__.py`, permitindo imports limpos entre eles.

Essa separação não é apenas estética – ela traz benefícios concretos de manutenção e extensibilidade. As configurações ficam isoladas em um único lugar usando `BaseSettings` do Pydantic, que sabe ler variáveis de ambiente automaticamente do arquivo `.env`. Os modelos definem contratos claros para entrada e saída de dados. E os services encapsulam a lógica de negócio, permitindo que sejam reutilizados por múltiplos endpoints sem duplicação de código. Quando um novo endpoint precisa ser adicionado, a estrutura já está pronta para absorver a mudança.

> *"Essa separacao vai deixar o codigo bem mais facil de manter e estender no futuro."*

Ao usar `BaseSettings` do Pydantic com `extra='allow'`, você evita que variáveis de ambiente não mapeadas nos campos da classe quebrem a aplicação na inicialização. Isso é especialmente útil quando o mesmo arquivo `.env` contém chaves de diferentes serviços – como Qdrant e Groq – e você quer adicionar novas integrações sem alterar a classe de configuração existente.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Embeddings como serviço isolado: encapsulando a geração de vetores

A geração de embeddings é o coração de qualquer sistema de busca semântica, e ela merece ser tratada como um serviço totalmente separado. O `EmbeddingService` encapsula a instanciação dos três modelos de embedding – dense (TextEmbedding), sparse (SparseTextEmbedding) e ColBERT (LateInteractionTextEmbedding) – e expõe um único método `embed_query` que recebe a consulta do usuário e retorna uma tupla com os três tipos de embedding.

Essa abordagem de isolamento traz vantagens significativas. Primeiro, o serviço de embeddings pode ser reutilizado por qualquer outro serviço que precise gerar vetores, sem duplicação de lógica. Segundo, a configuração dos modelos fica centralizada no `settings`, podendo ser alterada via variáveis de ambiente sem tocar no código. Terceiro, a lógica de conversão de formatos – `.tolist()` para dense e ColBERT, `.as_object()` para sparse – fica encapsulada em um único lugar, usando exatamente a mesma estratégia do script de ingestão original.

> *"Aqui eu vou encapsular toda a logica de geracao de embeddings em um servico totalmente isolado, totalmente separado."*

Ao criar um serviço de embeddings, mantenha a mesma lógica e os mesmos modelos usados na ingestão. Se os embeddings de busca forem gerados com modelos diferentes dos embeddings de ingestão, a qualidade da busca semântica será severamente prejudicada. A consistência entre ingestão e consulta é fundamental para resultados relevantes.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Busca híbrida em camadas: dense, sparse e ColBERT trabalhando juntos

O serviço de busca (`SearchService`) é o coração da aplicação e implementa uma estratégia híbrida em camadas que combina três tipos de embedding para maximizar a qualidade dos resultados. A arquitetura usa o mecanismo de `prefetch` do Qdrant para orquestrar essa combinação: primeiro, busca 20 resultados usando dense embeddings e outros 20 usando sparse embeddings, separadamente.

Esses dois conjuntos de resultados são combinados usando Reciprocal Rank Fusion (RRF), uma técnica que funde rankings de diferentes fontes levando em conta a posição relativa de cada resultado. O RRF retorna 15 resultados combinados. Por fim, o ColBERT (Late Interaction) é aplicado como filtro final sobre esses 15 resultados, considerando interações token a token para selecionar apenas os mais relevantes, retornando o número de resultados definido pelo usuário (3 por padrão). Dense captura similaridade semântica geral, sparse captura matches exatos de palavras-chave, e ColBERT faz o refinamento final.

> *"Essa estrategia em camadas garante que a gente aproveite o melhor de cada tipo de embedding. Dense captura similaridade semantica geral, o sparse captura matches de palavras-chave exatas, e ColBERT faz o refinamento final, considerando interacoes token a token."*

A normalização dos scores entre 0 e 1, dividindo cada score pelo score máximo encontrado, é uma prática recomendada ao retornar resultados para o cliente. Scores normalizados permitem que consumidores da API estabeleçam thresholds de relevância de forma

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

consistente, independentemente da escala interna do Qdrant.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

O endpoint de search: transformando busca vetorial em API acessível

O endpoint `/search` é o primeiro ponto de contato da aplicação com o mundo exterior. Ele recebe uma consulta em linguagem natural via `SearchRequest` (com campos `query` e `limit`), executa a busca híbrida no Qdrant e retorna um `SearchResponse` com uma lista de resultados contendo o score normalizado, o texto do chunk relevante e os metadados do documento de origem (ticker, company name, report date, form type).

A beleza desse endpoint está na simplicidade com que o FastAPI conecta todas as peças. O decorator `@app.post("/search", response_model=SearchResponse)` registra a rota, define o formato de resposta e gera a documentação automaticamente. A função handler simplesmente recebe um `SearchRequest` já validado pelo Pydantic e delega para o `SearchService`. Toda a complexidade da busca híbrida, geração de embeddings e comunicação com o Qdrant fica encapsulada nos services, enquanto o endpoint se preocupa apenas em receber a requisição e retornar a resposta.

> *"Qualquer sistema pode fazer requisicoes HTTP para o nosso endpoint de search e receber resultados de busca sobre os dados financeiros que foram ingeridos."*

Inclua sempre um endpoint raiz (`/`) como health check básico, retornando algo simples como `{"status": "online"}`. Isso permite verificar rapidamente se a API está no ar, sem precisar fazer uma busca real que depende de serviços externos como o Qdrant.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Do search ao RAG: adicionando geração de respostas com LLM

O salto do endpoint `/search` para o endpoint `/rag` representa a essência do padrão Retrieval-Augmented Generation. Em vez de apenas retornar os chunks brutos encontrados na busca, o endpoint `/rag` usa esses chunks como contexto para que um LLM (via Groq) gere uma resposta elaborada e compreensível. O `RagService` recebe o `SearchService` como dependência por injeção, reutilizando toda a infraestrutura de busca já construída.

O fluxo é direto: primeiro, o serviço executa a busca híbrida chamando `search_service.search()`. Depois, concatena os textos dos chunks recuperados em uma única string de contexto. Em seguida, monta um prompt com o contexto e a pergunta do usuário, e envia para a API da Groq com temperatura zero para respostas mais determinísticas. A resposta final inclui três campos: a pergunta original (`query`), a resposta gerada pelo LLM (`answer`) e uma lista de metadados dos chunks utilizados, incluindo os scores de relevância. Isso garante rastreabilidade completa da origem do contexto para a resposta.

> *"Eu costumo pensar nessa arquitetura como uma plataforma onde tudo gira em torno da minha ferramenta principal, que e a busca na minha base de conhecimento. E toda funcionalidade nova vira um novo endpoint."*

Ao construir o endpoint RAG, mantenha o endpoint de search disponível. Os dois endpoints compartilham a mesma lógica de busca, sem duplicação de código. Se o consumidor quer os chunks brutos para fazer seu próprio processamento, usa o `/search`. Se quer uma resposta elaborada pronta para o usuário final, usa o

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

`/rag`. Essa flexibilidade é uma marca de boa arquitetura.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Injeção de dependência entre services: composição em vez de acoplamento

Um dos padrões mais importantes demonstrados no curso é a forma como o `RagService` recebe o `SearchService` como dependência no construtor, em vez de instanciá-lo internamente. Essa abordagem de injeção de dependência garante que cada serviço tenha uma responsabilidade clara e bem delimitada: o `RagService` não precisa saber como conectar no Qdrant ou como gerar embeddings – ele só precisa usar o serviço de busca.

Essa composição se propaga por toda a arquitetura. O `SearchService` recebe URL, API key e collection name do Qdrant como parâmetros, instancia o `QdrantClient` e o `EmbeddingService` internamente. O `RagService` recebe o `SearchService` pronto e instancia o client da Groq. Cada camada adiciona sua própria complexidade sem contaminar as demais. O resultado é um sistema onde a lógica de busca, a lógica de geração de embeddings e a lógica de geração de respostas com LLM vivem em módulos independentes, conectados por interfaces claras.

> *"Isso e importante, porque em vez de criar uma nova instancia do SearchService aqui dentro, eu recebo ela pronta de fora. Isso facilita muito e mantem a responsabilidade."*

Quando um serviço depende de outro, prefira receber a dependência pronta no construtor em vez de criá-la internamente. Além de manter as responsabilidades separadas, essa abordagem facilita enormemente a escrita de testes: você pode passar um mock do `SearchService` para o `RagService` sem precisar de um Qdrant real rodando.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Routers como módulos plugáveis: refatorando para escalar

Conforme a aplicação cresce e novos endpoints são adicionados, o arquivo `main.py` tende a ficar poluído com imports, inicialização de serviços e definição de rotas, tudo misturado. A solução é usar routers do FastAPI – mini-aplicações que encapsulam seus próprios endpoints, dependências e lógica. Cada router vive em seu próprio arquivo dentro de um diretório `routers/`, e no final você só pluga eles no app principal com `app.include_router()`.

A refatoração é direta: todo o código relacionado ao endpoint `/search` vai para `routers/search.py`, e todo o código do endpoint `/rag` vai para `routers/rag.py`. Cada router instancia seus próprios serviços e define suas próprias rotas usando `APIRouter` em vez de `FastAPI`. O `main.py` fica com apenas três responsabilidades: instanciar o app, incluir os routers e definir o health check. Se amanhã você quiser adicionar um endpoint `/documents` ou um `/admin`, é só criar um novo router e plugar no main. A aplicação continua funcionando exatamente igual – a mudança é apenas organizacional.

> *"Routers no FastAPI sao como mini-aplicacoes que voce pode montar de forma modular. Cada router encapsula seus proprios endpoints, dependencias e logica. E, no final, voce so pluga eles no app principal."*

Ao extrair endpoints para routers, mantenha o endpoint raiz (health check) no `main.py`, pois ele não pertence a nenhum domínio específico. Use os routers exclusivamente para endpoints que representam funcionalidades de negócio – busca, RAG, ingestão, etc. Isso cria uma separação clara entre a infraestrutura da aplicação

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

e suas funcionalidades.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Prompt engineering desacoplado: separando instruções da lógica de negócio

Um dos últimos refinamentos da arquitetura é extrair o prompt do método `generateAnswer` para um arquivo de configuração separado (`config/prompts.py`). Pode parecer um detalhe pequeno, mas essa separação tem impacto significativo no dia a dia do desenvolvimento. Prompt engineering é uma atividade iterativa – você testa diferentes abordagens, ajusta instruções, adiciona exemplos few-shot, muda completamente a estratégia – e nada disso deveria exigir alterações no código do serviço.

Com o prompt vivendo em um arquivo dedicado como uma constante, o serviço simplesmente chama `RAG_PROMPT.format(context=con` `query=query)` e recebe a string completa pronta para enviar ao LLM. Se no futuro você precisar de outros prompts – para sumarização, análise de risco ou qualquer outra funcionalidade – todos ficam organizados em um único lugar. O código do serviço permanece limpo e focado na lógica de orquestração, enquanto o arquivo de prompts se torna o espaço natural para experimentação e refinamento.

> *"Prompt engineering e algo que voce itera bastante, testando diferentes abordagens, ate encontrar o que funciona melhor. Entao a solucao e extrair o prompt pra um arquivo de configuracao separado."*

Centralize todos os prompts da aplicação em um único arquivo de configuração. Além de facilitar a iteração, isso permite que diferentes membros da equipe trabalhem nos prompts sem precisar entender a lógica dos services. Em equipes maiores, o arquivo de prompts pode até ser versionado e revisado separada-

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

mente do código de negócio.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Resumo Final

A mensagem central do curso "Arquitetura RAG" é que construir uma aplicação de IA para produção não é apenas sobre fazer o modelo funcionar – é sobre criar uma arquitetura que seja modular, extensível e fácil de manter. Daniel Romero demonstra isso partindo de um pipeline de ingestão já funcional e construindo, passo a passo, uma API profissional que transforma busca semântica em um serviço acessível.

Os princípios que percorrem todo o curso são claros: separação de responsabilidades em camadas (config, models, services, routers), validação automática de dados com Pydantic, encapsulamento de lógica em serviços reutilizáveis, injeção de dependências entre componentes e desacoplamento de configurações e prompts do código de negócio. A estratégia de busca híbrida – combinando dense, sparse e ColBERT embeddings com Reciprocal Rank Fusion – mostra que a qualidade dos resultados depende de uma orquestração inteligente de múltiplas técnicas, não de uma solução única.

O padrão arquitetural mais poderoso apresentado é pensar na aplicação como uma plataforma onde tudo gira em torno da busca na base de conhecimento, e cada funcionalidade nova vira um novo endpoint. O `/search` retorna chunks brutos, o `/rag` gera respostas elaboradas com LLM, e novos endpoints podem ser adicionados sem alterar os existentes. Essa mentalidade de plataforma, aliada à refatoração constante, é o que separa protótipos de sistemas de IA prontos para produção.

Ao construir sua própria aplicação RAG, comece pelo endpoint de busca antes de adicionar a camada de geração com LLM. Ter a busca funcionando de forma isolada permite validar a qualidade

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

dos resultados recuperados antes de alimentá-los como contexto para o modelo. Se os chunks retornados não forem relevantes, nenhuma engenharia de prompt vai salvar a qualidade das respostas geradas.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Sobre o Curso

Arquitetura RAG é um curso ministrado por **Daniel Romero** que ensina como construir uma API de Retrieval-Augmented Generation (RAG) do zero, partindo de um pipeline de ingestão de dados financeiros e transformando-o em uma aplicação de produção com FastAPI, Qdrant e Groq.

O curso faz parte de uma trilha de engenharia de IA e abrange desde a introdução ao FastAPI até a construção de endpoints de busca semântica híbrida e geração de respostas com LLM, passando por refatoração e boas práticas de arquitetura de software aplicadas a projetos de IA.

Este blink foi gerado a partir das transcrições das aulas do curso.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00